Plano de implementação em Python

Versão revista, 27 de outubro de 2026. Substitui o plano anterior e incorpora as correções de segurança acordadas: pseudónimos por época, Gateway anónimo, chave de época pelo Kafka e veredito sem identidade.

1. Escolhas criptográficas

Função	Primitiva (`cryptography`)	Alternativa a comparar no relatório
Cifra do relatório, da corroboração e das chaves de época	AES-256-GCM, nonce aleatório de 12 bytes	ChaCha20-Poly1305
Assinaturas (credencial, relatório, corroboração, veredito, TSA)	Ed25519 (determinística)	ECDSA P-256, RSA-PSS
Cifra para um destinatário (cofre do Arbiter e `K_ep` por membro)	X25519 efémero + HKDF-SHA256 + AES-GCM (estilo ECIES)	RSA-OAEP, com limite de cerca de 190 bytes em RSA-2048 com SHA-256
Chaves de longa duração do membro	`sk_login` (Ed25519, autenticação) e chave de cifra X25519, registadas na inscrição	Uma só chave para tudo (rejeitada: mistura assinar com decifrar)
Chaves privadas em disco	`BestAvailableEncryption` com passphrase	keyring do sistema
Aleatoriedade	`secrets` e `os.urandom`	Nunca `random`

2. Formato das mensagens

Todas as assinaturas usam **separação de domínio** e codificação com prefixos de comprimento, para que uma assinatura de um tipo de mensagem não possa ser reutilizada noutro tipo nem noutra época.

Mensagem	O que é assinado	Por quem
Relatório	`FED-REPORT-v1`, `epoch_id`, `report_id`, SHA-256 do `ciphertext`	`sk_ep` do membro
Corroboração	`FED-CORROB-v1`, `epoch_id`, `report_id` alvo, SHA-256 do `ciphertext`	`sk_ep` do membro
Credencial	`FED-CRED-v1`, `epoch_id`, `pk_ep`, SHA-256 do cofre	Coordinator
Prova de posse	`FED-POP-v1`, `epoch_id`, desafio do Coordinator	`sk_ep` do membro
Veredito	`FED-VERDICT-v1`, `report_id`, resultado	Arbiter

- O `associated_data` do AES-GCM é o domínio, o `epoch_id` e o `report_id`.
- O **cofre** é `ECIES(chave de cifra do Arbiter, member_id + pk_ep + epoch_id)`, criado pelo Coordinator e assinado dentro da credencial. O membro não o consegue alterar.
- O envelope tem `epoch_id`, `ciphertext`, `signature`, a credencial e, depois do Gateway, o token da TSA. O conteúdo JSON dentro do `ciphertext` mantém exatamente a estrutura do enunciado.

3. Componentes

Membro (CLI em Python)

- Guarda `sk_login`, a chave de cifra, `sk_ep` e `K_ep`, cifradas em disco.
- Publica e corrobora via Tor, com circuito novo por mensagem e sem reutilizar sessões HTTP. Nunca usa o certificado de login no Gateway.
- **Verificador:** decifra, valida o JSON contra modelos `pydantic` com os valores permitidos do enunciado, verifica credencial, assinatura, token e época, e descarta o que for inválido ou repetido.
- A ordem de "quem reportou primeiro" vem da hora do token da TSA, não da ordem no Kafka.

Coordinator (FastAPI, PostgreSQL)

- Tabelas: membros (chave de login, chave de cifra, estado), épocas, credenciais emitidas com restrição única `(member_id, epoch_id)`, revogações.
- **Emite no máximo uma credencial por membro e por época** e exige prova de posse de `sk_ep`. Se o membro perder a chave, espera pela época seguinte.

- Gera `K_ep` (32 bytes aleatórios), cifra-a para a chave de cifra de cada membro ativo e publica no tópico `control`.
- Recebe do Arbiter a identidade do autor numa disputa e aplica a sanção ou a revogação.

Gateway anónimo (FastAPI + asyncio, `uvicorn --no-access-log`)

- Valida só a estrutura do envelope com `pydantic` (tamanhos, base64, campos, `extra=forbid`). **Não consegue validar o JSON do relatório, que está cifrado.**
- Verifica a assinatura do Coordinator na credencial, que a época é a atual, a assinatura do envelope, e anti-replay (`report_id` já visto e par `(report_id, pk_ep)` já usado numa corroboração).
- Aplica uma quota de mensagens por `pk_ep` e por época, para limitar spam.
- Pede o token à TSA, junta as mensagens em lotes, baralha a ordem, espera um tempo aleatório (`secrets`) e publica no Kafka. Não guarda IPs.

Arbiter (serviço isolado num contentor)

- Só aceita pedidos autenticados (TLS mútuo com o certificado de login) e com limite de pedidos por membro, para não funcionar como oráculo de desanonimização.
- Recebe relatório e credencial, verifica a assinatura do Coordinator e do relatório, decifra o cofre e envia a identidade **só ao Coordinator**.
- Publica em `control` um veredito assinado **sem identidade**. Ao requerente devolve apenas o número do caso.

TSA

- Serviço próprio com certificado emitido pela CA. Recebe só um hash e devolve um token assinado com hash, hora e número de série.
- Duas opções: token simplificado assinado com Ed25519, ou RFC 3161 completo com `openssl ts`. A decisão está em aberto.

CA privada (módulo offline `ca.py`)

- Gera a raiz e assina os CSRs dos serviços e os certificados de login dos membros. Nunca emite credenciais de publicação.
- O broker Kafka precisa de keystore PKCS12, que se gera a partir de PEM com `cryptography` ou OpenSSL.

Kafka

- Tópicos `reports`, `corroborations` e `control`, com uma partição cada e retenção longa, para os membros recuperarem o histórico.
- **ACLs:** só o Gateway escreve em `reports` e `corroborations`. Só o Coordinator e o Arbiter escrevem em `control`. Os membros só leem. Isto bloqueia spam e mantém o anonimato na publicação.
- A leitura usa o certificado de login, por isso o Kafka sabe quem lê, mas não quem escreveu o quê.

4. Fluxos principais

Credencial por época. O membro faz login por mTLS e envia `pk_ep`. O Coordinator responde com um desafio, o membro devolve a prova de posse, o Coordinator verifica que não está revogado e que ainda não recebeu credencial nessa época, cria o cofre e devolve a credencial. O membro pode pedir a da época seguinte um pouco antes de ela começar.

Publicar. O membro cria o JSON com `report_id` aleatório, cifra com `K_ep`, assina com `sk_ep` e envia ao Gateway pelo Tor. O Gateway valida, obtém o token, publica em lote e confirma pelo mesmo canal.

Corroborar. O membro verifica que a `pk_ep` não é a do autor, cifra e assina a corroboração, e envia ao Gateway. O Gateway rejeita um segundo envio da mesma `pk_ep` para o mesmo relatório. A corroboração só vale na mesma época do relatório. Os membros contam `pk_ep` distintas e válidas.

Disputa. Um membro autenticado envia o relatório ao Arbiter. O Arbiter abre o cofre, entrega a identidade ao Coordinator e publica o veredito sem identidade.

Revogação. O Coordinator marca o membro como revogado e abre logo uma nova época. Publica a nova `K_ep` só para os membros ativos, e o Gateway rejeita credenciais antigas.

5. Stack

Componente	Tecnologia
Criptografia	`cryptography`
Serviços HTTP	FastAPI + uvicorn
Validação	`pydantic`
Kafka no Gateway	`aiokafka` (assíncrono)

Componente	Tecnologia
Kafka no cliente e no Coordinator	`confluent-kafka`
Cliente HTTP com Tor	`httpx[socks]`, proxy `socks5h://127.0.0.1:9050`
Base de dados	SQLAlchemy + PostgreSQL
Implantação	Docker Compose, com redes separadas e o Arbiter isolado

6. Cuidados específicos de Python

- Não é possível apagar chaves da memória de forma fiável. Guardar as chaves privadas cifradas em disco e declarar a limitação.
- Usar `secrets` para toda a aleatoriedade e `hmac.compare_digest` nas comparações.
- Desativar logs de acesso no Gateway.
- Um serviço por contentor, sem partilha de volumes de chaves.

7. Ordem de construção

1. **Cripto e envelopes:** AES-GCM, Ed25519, ECIES, separação de domínio, com testes unitários.
2. **CA e Coordinator:** certificados, inscrição, credencial única por época com prova de posse.
3. **Gateway:** validação, anti-replay, quotas.
4. **Kafka e verificador do membro:** tópicos, ACLs, leitura com recuperação de histórico.
5. **Corroboração:** regra de uma por `pk_ep` e proibição de auto-corroboração.
6. **Revogação e épocas:** nova época imediata, distribuição de `K_ep`.
7. **Arbiter e disputas.**
8. **TSA e Tor**, por último, para não bloquear o resto.

Se o tempo apertar, os passos 1 a 5 já dão um sistema que cumpre confidencialidade, autenticidade, integridade e corroboração.

8. Limitações a declarar no relatório

- A unlinkability é por época: dentro de uma época a mesma `pk_ep` liga os relatórios do mesmo membro, e uma disputa revela todos os dessa época.
- O Coordinator conhece a ligação membro e pseudónimo na emissão.

- O anonimato de rede depende de o Tor ou relay funcionar. Sem isso, o Gateway vê os IPs.
- A corroboração prova que o membro a fez, não que observou o incidente.
- O anti-timing do Gateway é limitado com pouco tráfego.
- O Coordinator é um ponto único de falha.
- Sem zeroing fiável de memória em Python.

9. Decisões em aberto

- Duração da época.
- Formato da TSA (RFC 3161 completo ou simplificado).
- Como o Arbiter lê o relatório numa disputa (recebe `K_ep` ou o requerente fornece o texto).
- Tor ou relay próprio.
- O que acontece aos relatórios de um membro depois de revogado.
- Escala assumida (número de membros e frequência de relatórios).